ACADEMIC EARLY WARNING SYSTEM

QA Assessment
Complete Activity Pack

IT143 CO3 Major Assessment | Trello QAPMF

Tailored instructions, 20 task cards, five milestones, 40 test procedures, evidence templates, and presentation guide.

Before you begin
Replace team/member placeholders and working-day references with your actual details. Results and approvals must reflect work your group performs. This pack is a guide, not a completed test report.

Contents

1. Purpose and how to use this pack

2. Exact board content

3. Timeline and milestones

4. Copy-ready task cards

5. Test plan to attach to QA-02

6. Test cases and execution sheet

7. Evidence and defect templates

8. Final QA summary template

9. Presentation script and likely questions

10. Rubric and submission audit

Source documents

1. Purpose and how to use this pack

Project: Academic Early Warning System (AEWS) for Bukidnon State University.

This pack translates the supplied IT143 activity and its 25-point rubric into a Trello setup, task plan, test plan, test procedures, evidence templates, presentation, and submission checklist tailored to the local AEWS repository. It is a preparation pack, not evidence that testing has passed. No live system tests were performed for this document and no Trello board was created.

Replace [Team Name], [M1 Name] through [M5 Name], [Start Date], [Capstone Deadline], and [Submission Deadline]. Convert relative workdays into actual Trello due dates before submission. Days below are working days, not official assessment dates. If your deadline is shorter, adjust scope and dependencies openly rather than inventing earlier work.

Required by activity: existing Capstone group; board title and QA description; organized workflow; major QA task cards with descriptions, due dates, members, and applicable checklists; realistic timeline and milestones; labels, comments, attachments; public board link and board screenshot submitted individually by every member; no confidential data on the public board. The suggested lists and sample task names are examples in the activity, not a mandatory card count. Group-size wording is 'maximum of 4-5 members'; clarify only if necessary.

The rubric additionally scores presentation/explanation. Neither PDF specifies the deadline, submission portal, presentation format/duration, required number of test cases, a separate report, or performance thresholds. The detailed outputs and targets in this pack are recommendations to support the board; confirm assessment logistics with your instructor.

System basis

Reviewed: README.md; frontend/src/App.jsx; backend/app/routers/classes.py; backend/app/routers/amu_staff.py; backend/app/routers/students.py; backend/app/routers/admin.py; backend/app/ai_model.py; backend/MODEL_DOCUMENTATION.md; and backend/tests/test_amu_prediction_requirements.py. Repository inspection establishes test scope, not operational correctness. Requirements should be approved independently; current implementation is not automatically the correct acceptance standard.

AEWS uses a React frontend, FastAPI backend, MongoDB, and student-risk prediction components. Its workflows include instructor class management and academic uploads, AMU referrals/needs assessments/prediction/support routing, administrative accounts and reporting, and student login/profile/dashboard/assessment.

Important QA distinction: automatic referral logic in the reviewed class router checks whole midterm grades from 2.50 through 5.00 inclusive. This is distinct from ML risk prediction. The AMU prediction endpoint requires midterm grades, attendance, and a completed needs assessment before inference. The shared prediction helper has a separate sufficiency check. Test each entry point and resolve any mismatch with approved requirements. Do not claim every 2.50 grade must produce an ML High Risk result.

2. Exact board content

Title: **[Team Name] - Capstone 3 QA Project Management**

Description to paste:

This board manages quality assurance for our Academic Early Warning System (AEWS), which supports academic performance monitoring and early intervention at Bukidnon State University. QA covers instructor class management, classlist/gradesheet/attendance uploads, student information, referrals, needs assessments, risk prediction, support routing, administration, and reports. Our goals are correct academic-data processing, appropriate access control, traceable referral and prediction behavior, usable workflows, and reliable outputs. Each task has an owner, due date, checklist, expected deliverable, and review criteria. Work moves through Backlog, To Do, In Progress, Testing, and Completed. Completion requires reviewed evidence. Our timeline is aligned with [Capstone Deadline]. Public evidence uses synthetic or sanitized records only.

Lists, left to right:

1. Backlog: identified work not yet committed for immediate execution.

2. To Do: work ready to start with scope, owner, due date, and dependencies known.

3. In Progress: owner is actively doing the work.

4. Testing: execution evidence or deliverable is being validated/reviewed.

5. Completed: acceptance criteria met, checklist complete, evidence attached, reviewer recorded.

If a test finds a product defect, record the test result honestly and link a separate BUG card. The QA task may finish when planned execution/reporting is complete; unresolved defects must remain visible and cannot be interpreted as product approval. Failed deliverable review sends the card back to In Progress. Final QA approval requires the release criteria in section 5.

Labels: Functional Testing; Data Validation; Security and Privacy; AI Validation; Usability; Performance; Documentation; Defect; Milestone. Optional priority labels: Critical, High, Medium, Low. Use category labels on every applicable card and priority labels on defects. Lists track progress; labels describe the work.

Roles

Placeholder	Responsibility	Main cards
M1	QA lead, schedule, reviews, final summary	01, 02, 18, 19, 20
M2	Instructor workflow and academic-data tester	03, 06, 07, 08, 09
M3	AMU, assessment, prediction tester	10, 11, 12, 13
M4	Accounts, authorization, admin and student tester	04, 05, 14, 15
M5	Reports, usability, performance and regression support	16, 17, 18

With four members, divide M5 work among M1-M4 and reduce parallel workload accordingly. Each BUG goes to the developer responsible for the affected area; a tester reviews the fix. Replace placeholders with real Trello members. Roles do not imply exclusive ownership: everyone should contribute genuine updates and evidence.

3. Timeline and milestones

Start: [Start Date]. Capstone milestone: [Capstone Deadline]. Submission: [Submission Deadline].

Workdays	Work	Dependency	Milestone/deliverable
D1-D2	Scope, requirements, approved acceptance criteria	Project documents	M1: QA scope and test plan reviewed
D2-D4	Fixtures, environment, test cases	Requirements baseline	M2: test cases and test environment ready
D5-D7	Accounts, roles, instructor workflows, uploads	M2	Recorded first-wave results
D7-D10	Referrals, assessment, prediction, student/admin workflows	Required academic data ready	M3: planned initial execution recorded
D9-D12	Reports, usability, performance; fixes in parallel	Relevant features stable	Quality findings and defect register
D13-D15	Retest fixes and run regression/end-to-end flow	Fixes available	M4: retest and regression reviewed
D16	Final review and readiness decision	Results and defect status	M5: final QA decision
D17	Board audit, walkthrough, screenshots, individual submission	M5, accurate current board	Submission package

Create five milestone cards using the M1-M5 names above. Assign M1 as coordinator, dates D2/D4/D10/D15/D16, label Milestone, link prerequisite cards, and attach the corresponding reviewed output. A milestone is complete only when its output and review are recorded. Do not buy a timeline feature merely for this activity; due dates and milestone cards can show the plan.

4. Copy-ready task cards

For every card below: use the listed title; set an actual Trello due date corresponding to D#; assign the named real member; add the labels; paste objective/output/acceptance/dependencies into the description; turn checklist lines into Trello checklist items. Initial status is Backlog unless genuinely scheduled or already underway. Upload completed outputs as they become available, not fabricated completed documents.

QA-01 | Review AEWS requirements and risks

Owner M1; reviewer M4; due D2; labels Documentation, Data Validation.

Objective: establish approved behavior for all four roles and high-risk academic workflows.

Output: requirements/acceptance baseline and risk register.

Dependencies: project scope and Capstone schedule.

Acceptance: all scoped modules have requirement IDs, expected behavior, owner, and test mapping; uncertainties are recorded and resolved or explicitly deferred.

[ ] Inventory instructor, AMU, admin, student workflows.

[ ] Confirm grade scale, referral threshold, attendance interpretation, and prediction prerequisites.

[ ] Define role/record access matrix and public student-link rules.

[ ] Agree report formulas, performance targets, and release criteria.

[ ] Record privacy, upload, prediction, and integration risks; obtain review.

QA-02 | Prepare and approve AEWS test plan

Owner M1; reviewer M3; due D2; label Documentation.

Objective: define a feasible, traceable QA approach.

Output: completed section 5 adapted to the project, schedule, responsibilities.

Dependencies: QA-01.

Acceptance: scope, environment, entry/exit criteria, defect process, deliverables, and dates reviewed.

[ ] Set scope and exclusions.

[ ] Assign people and real dates.

[ ] Define evidence and severity rules.

[ ] Review milestones and fix/retest allowance.

[ ] Record reviewer and approval date.

QA-03 | Prepare environment, fixtures and test cases

Owner M2; reviewer M5; due D4; labels Documentation, Data Validation.

Objective: make execution repeatable without real student data.

Output: environment record, synthetic fixtures, reviewed test cases.

Dependencies: QA-01, QA-02.

Acceptance: four roles plus ownership-isolation fixtures are available; all planned cases have steps and expected results.

[ ] Record build/commit, browser, operating system, API and test database.

[ ] Prepare two instructors, AMU staff, admin and two students.

[ ] Prepare separate classes and valid/invalid academic files.

[ ] Review section 6 cases and add project-specific requirements.

[ ] Confirm test email sink or controlled recipients; keep credentials private.

QA-04 | Validate registration, approval and authentication

Owner M4; reviewer M1; due D6; labels Functional Testing, Security and Privacy.

Objective: verify account lifecycle and authentication decisions.

Output: TC-01 through TC-05 results.

Dependencies: QA-03.

Acceptance: each case executed or explicitly blocked; failures linked to BUG cards.

[ ] Test valid registration and invalid/duplicate input.

[ ] Test verification and pending/approved/rejected account behavior.

[ ] Test valid and invalid login.

[ ] Test password recovery and invalid/reused recovery token.

[ ] Test logout, session expiry, and record evidence.

QA-05 | Validate role and record access isolation

Owner M4; reviewer M2; due D7; label Security and Privacy.

Objective: prevent cross-role and cross-owner access through UI and API.

Output: TC-06 through TC-08 results and access matrix.

Dependencies: QA-03, QA-04.

Acceptance: unauthorized requests do not expose/modify protected records; security failures are escalated as release blockers.

[ ] Test no-session and expired-session access.

[ ] Test instructor access to admin/AMU operations.

[ ] Test instructor A requesting instructor B's class data and mutations.

[ ] Test student A requesting student B's data/profile changes.

[ ] Check server decisions, not only hidden navigation.

QA-06 | Validate class creation, archive and restore

Owner M2; reviewer M4; due D6; label Functional Testing.

Objective: verify class lifecycle and ownership.

Output: TC-09 and TC-10 evidence.

Dependencies: QA-03, QA-04.

Acceptance: expected lifecycle and related student-record behavior match approved requirements.

[ ] Create class with valid fields.

[ ] Submit missing/invalid fields.

[ ] Check correct owner's list.

[ ] Archive then restore a synthetic class.

[ ] Verify class and enrollment consistency.

QA-07 | Validate academic uploads and student matching

Owner M2; reviewer M3; due D7; labels Functional Testing, Data Validation.

Objective: verify accurate classlist/gradesheet/attendance ingestion and safe rejection.

Output: TC-11 through TC-15 results and input-to-record reconciliation.

Dependencies: QA-03, QA-06.

Acceptance: supported valid files map correctly; invalid data cannot silently corrupt or cross-match records.

[ ] Upload a valid supported classlist and compare identities/counts.

[ ] Upload supported grades and attendance formats.

[ ] Test mismatched IDs/names, duplicates, and re-upload behavior.

[ ] Test malformed, empty, unsupported and oversized files.

[ ] Check error feedback and persistent data after each rejection.

QA-08 | Validate grades and academic history

Owner M2; reviewer M3; due D7; label Data Validation.

Objective: verify whole grades, components and previous-term values remain distinguishable.

Output: TC-16, TC-17 results.

Dependencies: QA-07.

Acceptance: approved calculations/mappings and term separation match independent fixture expectations.

[ ] Reconcile midterm/final and component values.

[ ] Check previous midterm/final views.

[ ] Test missing values separately from zero.

[ ] Check invalid grade handling and precision.

[ ] Record before/after upload comparisons.

QA-09 | Validate attendance analytics

Owner M2; reviewer M5; due D7; label Data Validation.

Objective: verify attendance calculations and valid zero attendance.

Output: TC-18 results and manual calculation.

Dependencies: QA-07.

Acceptance: displayed totals/rates match approved calculations; missing and zero are distinguished.

[ ] Use known attendance counts.

[ ] Calculate expected rates independently.

[ ] Check 0%, 100%, missing and out-of-range input.

[ ] Compare attendance view and downstream record.

QA-10 | Validate automatic referral and AMU handoff

Owner M3; reviewer M2; due D9; labels Functional Testing, Data Validation.

Objective: verify whole-midterm threshold behavior and accurate referral routing.

Output: TC-19 through TC-21 results.

Dependencies: QA-07, QA-08.

Acceptance: boundary cases match approved rules; correct student/class/AMU referral appears without unintended duplicates.

[ ] Test grades 2.49, 2.50, 5.00 and 5.01 using allowed fixture preparation.

[ ] Verify whole midterm grade rather than individual component triggers.

[ ] Check referral reasons and student/class links.

[ ] Repeat update/upload and check duplication behavior.

[ ] Verify downstream AMU visibility and controlled notification behavior.

QA-11 | Validate needs-assessment lifecycle

Owner M3; reviewer M4; due D9; labels Functional Testing, Data Validation, Security and Privacy.

Objective: verify configuration, invitation, completion, validation and student linkage.

Output: TC-22 through TC-24 results.

Dependencies: QA-10, test form configuration.

Acceptance: required fields enforced; false/no answers accepted where valid; invalid links rejected; answers map to the correct record.

[ ] Configure a test form through an authorized admin.

[ ] Open a valid invitation and submit complete answers.

[ ] Try missing required answers, including conditional fields if present.

[ ] Test invalid/expired/reused invitation according to approved lifecycle.

[ ] Check upload path if used and reconcile stored answers.

QA-12 | Validate risk prediction and model traceability

Owner M3; reviewer M1; due D10; labels AI Validation, Data Validation.

Objective: verify input prerequisites, output integrity and model provenance without overstating accuracy.

Output: TC-25 through TC-28 results; model/version evaluation record.

Dependencies: QA-08, QA-09, QA-11.

Acceptance: missing-source AMU predictions are blocked; valid zeros/negative answers are handled correctly; outputs match the selected pipeline; model/fallback source and evaluation limitations recorded.

[ ] Remove each required source independently before AMU inference.

[ ] Test zero attendance and genuine false assessment responses.

[ ] Verify feature order/profile, label mapping and repeatability for fixed inputs/version.

[ ] Test missing-model behavior in isolated configuration.

[ ] Review held-out metrics, confusion matrix and High Risk recall; distinguish stored metrics from new execution.

[ ] Compare instructor and AMU prediction entry-point requirements against approved rules.

QA-13 | Validate AMU support routing and communications

Owner M3; reviewer M5; due D10; label Functional Testing.

Objective: verify intervention decisions and notifications attach to the correct student.

Output: TC-29 and TC-30 results.

Dependencies: QA-10, QA-11.

Acceptance: routing persists correctly and controlled sends/errors have verifiable outcomes.

[ ] Save and reopen support routing for a synthetic referral.

[ ] Check assigned service/action details.

[ ] Send only to controlled test recipients.

[ ] Test delivery/queue failure feedback in an isolated test setup.

[ ] Verify correct recipient and absence of duplicate unintended actions.

QA-14 | Validate student dashboard and profile

Owner M4; reviewer M3; due D10; labels Functional Testing, Security and Privacy.

Objective: verify the student's own information and profile lifecycle.

Output: TC-31 and TC-32 results.

Dependencies: QA-04, QA-10.

Acceptance: correct academic/referral information appears; profile updates validate and persist under approved identity checks.

[ ] Test student login and required profile completion.

[ ] Compare dashboard against synthetic source records.

[ ] Update allowed profile fields and reload.

[ ] Check invalid inputs and cross-student access with QA-05.

QA-15 | Validate administrative accounts and form management

Owner M4; reviewer M1; due D10; labels Functional Testing, Security and Privacy.

Objective: verify admin account decisions, archive/restore and assessment configuration.

Output: TC-33 and TC-34 results.

Dependencies: QA-03, QA-04.

Acceptance: authorized actions persist and unauthorized changes fail; form changes follow approved behavior for existing invitations.

[ ] Approve/decline synthetic pending accounts.

[ ] Archive/restore a test account and check login behavior.

[ ] Edit needs-assessment form and check new invitation.

[ ] Check existing invitation behavior after configuration changes.

[ ] Verify audit entries for actions intended to be logged.

QA-16 | Validate dashboards, reports and exports

Owner M5; reviewer M2; due D12; labels Functional Testing, Data Validation.

Objective: reconcile dashboard/report figures and exported content.

Output: TC-35 and TC-36 results, sanitized sample exports.

Dependencies: completed synthetic data and referral fixtures.

Acceptance: totals, filters, row scope, labels and formats reconcile with expected records; no unauthorized records appear.

[ ] Reconcile instructor, AMU and admin totals separately.

[ ] Check filters, search and empty results.

[ ] Export available formats for each role.

[ ] Verify titles, headers, page breaks and long values.

[ ] Compare export rows/counts to filtered source data.

QA-17 | Evaluate usability, responsiveness and performance

Owner M5; reviewer M4; due D12; labels Usability, Performance.

Objective: measure representative tasks and responsiveness under defined conditions.

Output: TC-37 through TC-39 results and timing/usability notes.

Dependencies: stable core workflows, agreed targets from QA-01.

Acceptance: measured results compared to agreed targets; environment, data volume, method and limitations recorded.

[ ] Complete upload/referral/assessment workflows using written tasks.

[ ] Check keyboard access, validation, loading/error states and narrow screens.

[ ] Measure dashboard/search/prediction response times under repeatable conditions.

[ ] Record failures and prioritize remediation.

QA-18 | Fix verification, regression and end-to-end test

Owner M5, M1 coordinates; reviewer M3; due D15; labels Functional Testing, Data Validation.

Objective: verify fixes and prevent regressions across the academic intervention workflow.

Output: TC-40 results, retest register, regression summary.

Dependencies: initial execution and available defect fixes.

Acceptance: original failed steps and affected regression cases rerun on identified build; remaining issues visible.

[ ] Link each fix/build to its original defect.

[ ] Repeat failed case and boundary variants.

[ ] Run upload -> referral -> assessment -> prediction -> routing -> report flow.

[ ] Recheck role isolation after related changes.

[ ] Reopen failed fixes and summarize unresolved issues.

QA-19 | Final QA review and readiness decision

Owner M1; reviewer whole team; due D16; labels Documentation, Milestone.

Objective: make an evidence-based readiness decision.

Output: final QA summary and actual reviewer decision.

Dependencies: QA-04 through QA-18 results.

Acceptance: test coverage, open defects, model limitations and exit criteria reviewed; approval is recorded only if actually given.

[ ] Reconcile planned/executed/passed/failed/blocked/not-run counts.

[ ] Review all open defects and release blockers.

[ ] Review requirements coverage and model evidence.

[ ] Record Ready / Ready with accepted limitations / Not ready, reviewer and date.

[ ] Attach summary and link relevant evidence.

QA-20 | Audit board, present and prepare individual submissions

Owner M1; all members participate; reviewer M4; due D17; label Documentation.

Objective: satisfy submission and explanation requirements.

Output: accurate public board, readable screenshots, walkthrough, individual submissions.

Dependencies: current results and QA-19 decision (including Not ready if applicable).

Acceptance: each member submits public link and screenshot; no confidential information exposed; explanation matches actual status.

[ ] Replace placeholders and relative dates.

[ ] Check fields, labels, comments, evidence and true status on each card.

[ ] Sanitize descriptions, attachments and screenshots before public visibility.

[ ] Verify public link while signed out.

[ ] Rehearse section 9; capture board and optional detail screenshots.

[ ] Each member submits individually and confirms their own submission.

5. Test plan to attach to QA-02

Objective: verify AEWS supports accurate academic monitoring and traceable intervention while protecting role/record boundaries.

Scope: accounts/session; role and ownership checks; classes; supported classlist, gradesheet and attendance imports; academic history; referral threshold and AMU handoff; needs assessment; prediction prerequisites and provenance; routing/communication; student profile/dashboard; administration; dashboards/reports/exports; usability and baseline performance.

Exclusions unless separately scheduled: production penetration testing, university-wide load certification, proof of clinical/student-outcome effectiveness, and accuracy claims unsupported by independent evaluation. Record any additional exclusions with reasons.

Approach: requirements review, positive/negative and boundary tests, UI/API integration tests, fixture reconciliation, targeted existing automated tests, manual usability review, performance measurement, defect retesting, and end-to-end regression. Treat an existing automated suite as supporting evidence only after executing it on the recorded build.

Environment: [Commit/build], [test URL], [API version], [browser/version], [OS], [test MongoDB database], [model artifact/version], [test email sink]. Keep secret values and access credentials off Trello. Use an isolated test database for archive/delete, failure injection, and invalid data fixtures.

Entry criteria: approved scope/expected behavior; runnable build; controlled test accounts; fixture files; accessible environment; no real confidential data in evidence; cases reviewed.

Proposed exit criteria (team recommendations, not instructor mandates): all scoped cases executed or explicitly accounted for; no unresolved Critical/High defects affecting identity, record isolation, grade/attendance integrity or core intervention workflow; core end-to-end cases pass; remaining lower-severity issues have documented impact and authorized acceptance; final summary and evidence reviewed. Do not silently waive a blocker to claim completion.

Proposed performance baseline: with a synthetic 100-student class and recorded environment, ordinary dashboard/search interactions finish within 3 seconds and single-student prediction within 5 seconds under a single-user baseline. These are starting targets to agree in QA-01, not measured results or promises. Record 20 repetitions, cold/warm conditions and median/p95; report upload time separately with file size/rows. Expand concurrency only if scoped and supported.

Model QA: record dataset provenance, model/profile/version and feature order. Review train/test separation, duplicate/student leakage and whether outcome-derived features leak target labels. Report held-out sample counts, class distribution, confusion matrix and High Risk recall alongside accuracy/F1. Do not equate confidence with accuracy or synthetic-data scores with real-world effectiveness. No numerical accuracy threshold is imposed by the assessment PDFs; agree one if the Capstone requires it.

Severity: Critical = unauthorized sensitive-data access, major data loss/corruption or total core outage; High = core workflow incorrect/unavailable with no acceptable workaround; Medium = partial failure with workaround; Low = minor display/text issue. Severity describes impact; priority describes urgency.

Definition of done: task output exists; acceptance criteria reviewed; checklist complete; actual results recorded; defects linked; evidence sanitized; reviewer/date recorded; status accurate.

6. Test cases and execution sheet

Common setup: use only isolated synthetic records. Create Instructor A and B, Admin A, AMU A (and another AMU/college if routing scope requires), Student A and B; Class A belongs to Instructor A and Class B to Instructor B. Use approved test identities and controlled email addresses. Prepare five roster records with valid project-format IDs; maintain a fixture manifest privately. Expand expected values after QA-01 approves the business rules. Every test starts Not run; expected behavior below is a proposed oracle, not a claimed result.

For each case record: requirement ID; case ID; build; environment; preconditions; exact input; steps; expected result; actual result; Pass/Fail/Blocked/Not run; tester; date; evidence; defect ID. Split multi-scenario rows into separate executions when results differ.

ID / requirement	Preconditions and procedure	Expected result
TC-01 / R01 Accounts	Register valid synthetic staff details; verify through controlled email; inspect pending state.	Valid request saved once; verification and pending-approval states follow approved lifecycle; no premature privileged access.
TC-02 / R01	Repeat registration with existing email; separately omit required field and use invalid email/college.	Clear validation; no unintended duplicate account or partial privileged account.
TC-03 / R01	Admin approves one request and rejects another; attempt login for approved, pending and rejected users.	Access follows the approved state rules; state persists after reload.
TC-04 / R02 Session	Login with valid password, wrong password, then sign out and replay protected request; also test expired session.	Valid login works; wrong/expired/signed-out sessions cannot use protected resources.
TC-05 / R02	Request reset to controlled mailbox; use valid reset once, then invalid/expired/reused token; try old/new passwords.	Approved reset lifecycle enforced; valid reset changes credential; invalid tokens fail without exposing secrets.
TC-06 / R03 Access	Request protected staff pages and APIs without session; request admin mutation as instructor.	Server denies unauthorized operation; no data or state change. Public exceptions require documented scope.
TC-07 / R03	Instructor A reads, edits, uploads to and archives Class B by changing class/instructor identifiers in requests.	Cross-owner access denied for every operation; Class B unchanged.
TC-08 / R03	With Student A context, substitute Student B identifier in dashboard/profile requests; repeat without valid student identity proof.	Access/change follows approved identity policy; Student B's private information is not exposed or changed.
TC-09 / R04 Classes	Create Class A with valid required fields; separately omit required values; reload instructor list.	Correct class appears for correct owner; invalid creation fails clearly.
TC-10 / R04	Archive then restore a synthetic class; inspect active/archived lists and associated enrollments after each step.	Lifecycle and related records match approved archive rules; no loss or duplicate enrollments.
TC-11 / R05 Upload	Upload valid five-student classlist using supported template; reconcile every identity and displayed total.	Correct five records mapped once; preview/persisted values agree with source.
TC-12 / R05	Upload valid gradesheet and attendance for known roster; compare selected source cells to stored/displayed values.	Correct student/term/field mapping with no shift, unintended overwrite or missing rows.
TC-13 / R05	Upload file with mismatched student ID/name; separately repeat upload and introduce duplicate row.	Approved reject/update policy enforced; no cross-student writes or unintended duplicate record; partial handling reported clearly.

ID / requirement	Preconditions and procedure	Expected result
TC-14 / R05	Upload empty file, malformed supported file and unsupported MIME type in isolated setup.	Rejected clearly without crash or unintended data changes.
TC-15 / R05	Test file sizes at and just above 10 MiB; where endpoint supports multiple files, test aggregate at/above 50 MiB. Ensure fixture is otherwise valid.	Reviewed class-router limits allow boundary size validation and reject above-limit requests; record parsing errors separately from size checks. Other endpoints verified against their own rules.
TC-16 / R06 Grades	Import known midterm/final/component and previous-term values; inspect respective views and calculate approved derived totals independently.	Terms/components remain distinct; calculations and rounding match approved formulas.
TC-17 / R06	Enter/import missing grade, zero sentinel if applicable, allowed extremes and invalid values; reload.	Missing/out-of-range values handled explicitly; no false valid grade, corruption or silent interpretation.
TC-18 / R07 Attendance	Use approved session-count fixture (e.g. 8 present of 10 when this is the chosen formula); test 0%, 100%, missing and above 100%.	Expected 80% for simple fixture; approved attendance rules applied; zero valid and missing distinct; invalid range handled.
TC-19 / R08 Referral	Use separate eligible synthetic records for whole midterm 2.49, 2.50 and 5.00; use 5.01 only as an invalid-input test. Trigger supported save/upload flow.	Under reviewed threshold, 2.49 does not trigger this grade-based reason; 2.50/5.00 do; 5.01 rejected or not treated as valid trigger. Other approved reasons assessed independently.
TC-20 / R08	Hold whole midterm below referral threshold while varying CS/LAB/MO components; then change whole midterm to threshold.	Grade-based automatic referral follows whole midterm rule, not an individual component; confirm against approved policy.
TC-21 / R08	Repeat eligible upload/update; view AMU referral, class/student/reason and assigned staff scope.	Correct linked referral and permitted AMU visibility; no unintended duplicate or misrouting.
TC-22 / R09 Assessment	Create invitation for Student A; open valid link; answer every required field; submit; inspect AMU record.	Correct form/student; answers persist and are linked to Student A; success shown.
TC-23 / R09	Leave required field empty; separately answer valid boolean question false/no and complete other required fields.	Missing answer rejected; legitimate false/no not mistaken for missing.
TC-24 / R09	Open random invalid token; test expired and reused token where defined by approved lifecycle; check no other student's answers exposed.	Invalid access denied; expiry/reuse policy enforced without data leakage.
TC-25 / R10 Prediction	AMU predict with complete fixture; separately remove midterm, attendance, and genuine assessment answers.	Complete fixture can proceed; each missing-source case returns insufficient data and no prediction; model not invoked for incomplete cases.
TC-26 / R10	Set attendance to 0 with valid midterm and complete genuine assessment containing false answers; separately leave only derived assessment scores.	Valid zero/false data accepted as present; derived scores alone do not count as completed assessment.

ID / requirement	Preconditions and procedure	Expected result
TC-27 / R10	Record artifact/profile/feature order; repeat fixed complete input; compare API and UI result, label, probability scale and model source.	Stable result for fixed version/input; class mapping and percent scale correct; displayed explanation matches actual inputs; no promised label based solely on referral threshold.
TC-28 / R10	In isolated config remove model availability and restart/clear cache as required; predict complete fixture; compare instructor and AMU entry-point prerequisites.	Fallback/unavailability handled according to approved requirements; provenance not represented as trained-model output; prerequisite differences documented and resolved.
TC-29 / R11 Routing	Save support-routing action for Student A referral; reload and inspect Student B referral.	Correct action persists only for intended referral/student; required details validated.
TC-30 / R11 Communication	Send invitation/notice to controlled recipient; inspect receipt or test sink; simulate configured delivery/queue failure.	Correct recipient/content/link; queued and delivered states distinguished; failure is observable and retry policy does not cause unintended duplicates.
TC-31 / R12 Student	Login Student A, complete required profile if prompted, open dashboard and compare known data.	Correct own profile/academic/referral information; no Student B information.
TC-32 / R12	Update allowed profile field; reload; submit invalid value and attempt another student's identifier.	Valid own update persists; invalid/unauthorized update rejected.
TC-33 / R13 Admin	Admin approve/decline requests, archive/restore a synthetic user; test resulting account access.	Correct state changes and account access; non-admin cannot perform same mutations.
TC-34 / R13	Edit assessment form; generate new invitation; open older invitation; inspect intended activity log.	New/old forms follow approved versioning behavior; intended audit entry has correct actor/action/time without secrets.
TC-35 / R14 Reports	With known fixture totals, open instructor/AMU/admin dashboards; apply search and filters, including empty result.	Counts and scope match independently calculated records; correct handling of empty state.
TC-36 / R14	Export available report formats; compare rows/counts/filters; open output with long names and multi-page data.	Correct authorized scope/content, readable headings/page layout and no clipped critical information.
TC-37 / R15 Usability	Another member follows upload -> referral -> assessment task without coaching; record confusion and errors.	Task can be completed with understandable feedback; observations recorded, not fabricated user satisfaction scores.
TC-38 / R15	Navigate key forms with keyboard; check focus/error labels, narrow/mobile and desktop viewports, loading/network-error states.	Important controls reachable and legible; no hidden required actions; clear progress and recoverable errors.
TC-39 / R16 Performance	Record environment, 100 synthetic students, cold/warm conditions; time 20 dashboard/search/prediction runs using browser/network tools.	Results compared with agreed targets; median/p95, failures and limitations documented.
TC-40 / R17 Integration	On recorded build: roster -> grades/attendance -> threshold referral -> assessment -> prediction -> routing -> report; rerun affected cases for every fix.	Correct identities/data through all stages; original defects retested; no affected regression failure; remaining blockers documented.

Requirement IDs R01-R17 above are proposed QA traceability IDs, not official IDs from a signed requirements specification. Replace or map them to your real specification. Test every supported template/role variant actually in scope; 40 grouped cases are a starting suite, not proof of exhaustive coverage.

Supporting automated checks already represented in this repository

Backend test files include authentication, authorization, router helpers, AMU prediction requirements, errors, rate limiting, report letterheads and dashboard performance. Frontend files include upload validation, report export, idle session and AMU settings tests. These names show intended coverage; their presence does not establish that they pass.

From a correctly configured development environment, use the project's approved setup and run:

```
# From repository root, with backend environment activated
python -m pytest backend/tests
# Then, in a separate command/session from frontend directory
npm test
npm run build
```

Save output with build/date/environment and link failures to defects. Resolve setup failures separately from product failures. Do not post raw logs containing secrets. Manual integration/security/business-rule tests remain necessary even if these checks pass.

7. Evidence and defect templates

Test execution record

```
Case ID / requirement:
Build / environment:
Tester / date:
Preconditions and fixture:
Exact steps / input:
Expected result:
Actual result: [complete only after execution]
Status: Not run
Evidence attachment:
Defect link:
Reviewer / review date:
```

BUG card

```
Title: BUG-[number] | [Module] | [Observable failure]
Severity / priority:
Build / environment:
Linked case / requirement:
Preconditions:
Steps to reproduce:
1.
2.
Expected:
Actual:
Impact:
Sanitized evidence:
Assigned developer / due date:
Fix build / change reference:
Retest steps and actual result:
Regression cases:
Tester / reviewer / dates:
Resolution: Open / Fixed awaiting verification / Reopened / Verified
```

Defect checklist: reproduce; classify; assign; investigate; implement fix; record build; retest original steps; run affected regression; reviewer verifies; close or reopen. Add label Defect and module/type/priority labels. Use the same workflow lists: new triaged defects To Do, active fixes In Progress, fixes awaiting verification Testing, verified fixes Completed. Record accepted unresolved limitations explicitly; do not call them verified fixes.

Progress comment examples to adapt only when true

Planning: "Prepared [number] cases for academic uploads. Waiting for reviewed classlist template before execution. Next action: [owner/action/date]."

Execution: "Build [id]: executed [n] cases; [p] passed, [f] failed, [b] blocked. Linked BUG-[id]. Sanitized results attached. Next action: [action/date]."

Review: "Reviewed [deliverable] against [criteria]. Findings: [details]. Decision: [accepted/revision needed]. Reviewer: [name], [date]."

Retest: "Retested BUG-[id] on build [id]. Original steps: [actual result]. Regression cases [IDs]: [results]. Decision: [verified/reopened]. Evidence: [link]."

Evidence organization

Suggested attachment names: AEWS_QA_TestPlan_v1; AEWS_Requirements_TestMatrix_v1; AEWS_TestResults_[build]; AEWS_DefectRegister_[date]; AEWS_ModelValidation_[version]; AEWS_FinalQASummary_[date]. These can be simple documents/spreadsheets your group maintains; no specific format is mandated by the PDFs.

Use screenshots such as TC-019_referral_boundary_[build].png with readable relevant context. Keep test date/build and expected/actual result in the record. Redact real student identities, grades, assessment answers, emails, tokens, passwords, database connection strings and internal credentials. Never attach .env files or production datasets to a public board.

Risk register starter

Risk	Impact	Mitigation / owner
Upload maps to wrong student/term	Incorrect interventions	Identity/term fixture reconciliation; M2
Threshold boundary or grade-scale error	Wrong automatic referral	Independent boundary oracle and whole-grade tests; M3
Missing data treated as valid prediction input	Misleading risk result	AMU source-prerequisite and zero/false tests; M3
Cross-role/record exposure	Confidential information disclosure	UI and API ownership matrix; M4
Metrics/fallback represented as proven accuracy	Unsupported claims	Model provenance, holdout review and limitations; M3/M1
Integration fix breaks downstream reports	Inconsistent intervention history	End-to-end regression; M5
Schedule leaves no retest time	Unverified fixes	D13-D15 reserve and early triage; M1

8. Final QA summary template

```
Project: Academic Early Warning System
Team / build / environment / model version:
QA period:
Scope tested / excluded:
Total planned cases:
Passed:
Failed:
Blocked:
Not run:
Check: planned = passed + failed + blocked + not run
Execution rate = (passed + failed) / planned x 100
Pass rate among executed = passed / (passed + failed) x 100
Use N/A when denominator is zero; report blocked/not-run separately.
Requirements covered / uncovered:
Open defects by severity:
Critical/High blocker details:
Performance measured results and environment:
Model evaluation evidence and limitations:
Retest/regression outcome:
Residual risks and any accepted limitations, accepted by whom/date:
Decision: [Ready / Ready with accepted limitations / Not ready]
Decision rationale against exit criteria:
Reviewer names / dates / actual approval:
Evidence links:
```

9. Presentation script and likely questions

Suggested 4-5 minute walkthrough; duration is a recommendation only. Replace placeholders and report actual status. With four people, combine parts 4 and 5.

Member 1 - project and goals: "Our project is the Academic Early Warning System. It supports academic monitoring and early intervention through instructor, AMU, administrator and student workflows. This QA board organizes the work needed to verify academic-data accuracy, appropriate access, referral and prediction behavior, and reliable reporting. Our schedule is aligned with [actual Capstone milestone]."

Member 2 - structure and tasks: "Our lists are Backlog, To Do, In Progress, Testing and Completed. They show the actual state of work. This upload-testing card identifies the owner, deadline, inputs, checklist and expected evidence. We test valid files and invalid cases because a successful upload must also match the correct student and term. Completed means the task output has been reviewed, with product failures tracked separately as defects."

Member 3 - AEWS-specific QA: "We test the automatic referral boundary at whole midterm grade 2.50 through 5.00 according to our approved requirement. Referral is different from ML prediction. For the AMU prediction workflow, we verify the required grades, attendance and completed assessment. We also test valid zero attendance and false assessment answers. Model provenance and evaluation evidence support our explanation; we do not claim test accuracy from a screenshot of a prediction."

Member 4 - features and evidence: "Labels identify the testing category. Comments record actual progress, blockers and review decisions. Attachments provide sanitized plans, test results and defect evidence. We check role and ownership restrictions through the API as well as the interface, and keep real student records and credentials off the public board."

Member 5 - timeline and status: "Our milestones cover planning, test-case readiness, initial execution, retesting and final QA review. Fixing and retesting have reserved time. As of [date], [actual counts] cases are [statuses], with [actual open defects]. Our readiness decision is [actual decision] because [evidence]. Every member will submit the public board link and screenshot individually."

Likely question	Answer to adapt
Why Trello?	It makes tasks, ownership, due dates, discussion, evidence and current status visible in one shared workflow.
How does this demonstrate QA?	We define expected behavior before execution, record evidence, investigate failures, verify fixes and review readiness.
Why test 2.50?	It is an included boundary in the reviewed automatic referral rule; we also check values below and at the upper boundary.
Is a referral a High Risk prediction?	No. Referral is a workflow/business-rule action; the trained-model output is evaluated separately.
How do you know the system passed?	By recorded expected-versus-actual results on a named build, reviewed evidence and exit criteria; the board itself is not proof of success.
What happens when a test fails?	We log a linked defect, assign and prioritize it, fix it, retest and run affected regression.
Is the model accurate?	State only verified evaluation results with dataset/model version and limitations; otherwise say evaluation is pending.
Why not put every card in Completed?	Current status must be truthful, and final approval depends on evidence and unresolved blockers.

10. Rubric and submission audit

Rubric category	Maximum	Evidence to show
Board setup and organization	5	Required title; professional description; logical, understandable lists
Task detail and completeness	5	Actual task descriptions, assigned members, dates, meaningful checklists
Timeline and deadlines	5	Realistic dates, dependency order, visible milestones aligned to Capstone
Effective Trello features	5	Consistent relevant labels, genuine progress/review comments, useful attachments
Presentation and explanation	5	Clear walkthrough explaining structure, choices, timeline and accurate QA status
Total	25	No score guarantee; instructor evaluates the actual submission

Do these actions in order

1. Fill team/member details, actual dates and approved acceptance rules.

2. Create board and invite members; paste title/description and create the five lists.

3. Create labels, 20 task cards and five milestone cards from this pack. Merge QA-19 and final milestone if desired to avoid duplicate tracking; no fixed card count is required.

4. Set real owners and dates using Trello fields, not only text; paste checklists and dependencies.

5. Review and attach the adapted test plan, requirements mapping and test procedures.

6. Execute work in the test environment; enter real results, comments and sanitized evidence; create linked BUG cards for failures.

7. Fix, retest and run regression; record current readiness honestly.

8. Audit every card for missing fields and inaccurate status. Do not manufacture work to fill every list.

9. Sanitize the entire board and attachment history before making it public. Verify public visibility and the copied board URL from a signed-out/private browser context. If public access is prohibited by institutional policy, obtain the instructor's approved alternative rather than posting restricted data.

10. Capture a readable overview with board title, lists and cards. If one screenshot cannot show enough detail, add supplementary views and an opened representative card; the activity only explicitly asks for a board screenshot.

11. Rehearse the walkthrough and confirm presentation format, deadline and submission portal with instructor announcements.

12. EVERY member independently submits the same public group board link plus board screenshot through the required portal. One group submission does not replace individual submissions.

Submission text:

```
IT143 CO3 Major Assessment
Name: [Individual name]
Team: [Team Name]
Project: Academic Early Warning System (AEWS)
Public Trello board: [verified public URL]
```

```
Attachment: [readable board screenshot]
```

Optional individual confirmation checklist: link opens signed out; screenshot attached; correct name/team; submitted through required portal; submission receipt checked. Keep any private receipt out of the public board.

Source documents

- C:/Users/Ian/Downloads/IT143_CO3_MajorAssessment_Guidelines.pdf (three pages).

- C:/Users/Ian/Downloads/Documents/Rubric for Group Activity using Trello.pdf (one page).

The source PDFs govern the assessment. The schedule, roles, case IDs, targets, templates and card breakdown in this pack are tailored recommendations, to be adapted to approved AEWS requirements and actual project status.